Persistent and Fault-Tolerant UAV-IoT Coverage: A Multi-Agent Reinforcement Learning Framework for Joint Coverage, Connectivity, and Energy Management

Himanshu Nagwanshi
CS24BT055@iitdh.ac.in

Surya Prakash S
CS23BT068@iitdh.ac.in

Harsh Varshney
cs23bt004@iitdh.ac.in

Gaddam Viswaijeth
CS23BT076@iitdh.ac.in

Department of Computer Science and Engineering
Indian Institute of Technology Dharwad, Dharwad, India

***Abstract*—UAV swarms can provide temporary wireless coverage for Internet-of-Things (IoT) deployments in areas where fixed infrastructure is unavailable, damaged, or insufficient. Persistent operation is considerably harder than one-shot deployment because coverage, inter-UAV connectivity, battery depletion, limited charging resources, and UAV failures are tightly coupled. This paper formulates the problem of maintaining at least K UAVs over every ground sensor while preserving a connected swarm and respecting finite energy and safety constraints, and models the resulting decision process as a cooperative Decentralized Partially Observable Markov Decision Process (Dec-POMDP). We argue that the evaluation methodology is itself the unsolved part of this problem: a coverage ratio reported without an achievability reference cannot distinguish a strong controller from an infeasible configuration, and the widely used service-continuity metric is identically zero whenever the configuration is infeasible. We therefore introduce a *feasibility-certified evaluation framework* that (i) derives a rigorous lower bound on the number of UAVs required for K-coverage of a bounded region, (ii) supplies a teleporting placement oracle as an achievability ceiling against which any policy is normalised, and (iii) replaces the degenerate joint-constraint metric with non-degenerate time-averaged and refresh-time metrics. The bound is applied to a published configuration and shown to be provably infeasible, which also demonstrates that the conventional area-surplus criterion does not certify feasibility. On the corrected configuration the framework is paired with a Multi-Agent Proximal Policy Optimization controller under centralized training and decentralized execution, with parameter-shared actors, an explicit H-step connectivity feasibility mask, a reserve-UAV recovery mechanism, charging arbitration, and PHY-derived links in place of abstract coverage discs. Across five training seeds the controller attains $CK = 0.930 \pm 0.051$ persistent K-coverage against 0.850 for three rule-based baselines, maintains full connectivity, and is the only controller in the comparison to satisfy coverage and connectivity simultaneously ($SC = 0.200$ against 0.000); the reserve mechanism contributes $+0.058$ CK. Every reported quantity is recomputed by an automated claim audit, and the physical-layer model is shown to be identical, to within 0.012 dB, to the path-loss model of an independent standards-faithful network simulator.**

***Index Terms*—UAV-IoT networks, persistent coverage, fault tolerance, multi-agent reinforcement learning, MAPPO, Dec-POMDP, connectivity maintenance, feasibility bounds, evaluation methodology.**

I. Introduction

The Internet of Things (IoT) increasingly operates in environments such as agricultural fields, disaster-affected regions, industrial sites, and remote monitoring areas where fixed communication infrastructure may be sparse or temporarily unavailable. UAVs can provide rapidly deployable aerial communication or sensing support because their positions can be changed in response to the spatial distribution of demand. Energy-efficient UAV control has therefore been studied for communication coverage, fairness, and connectivity, while aerial-base-station placement has been formulated as a learning-based spatial optimization problem [1], [2].

The practical difficulty changes substantially when the network must remain operational for a long mission rather than only achieve good coverage at one instant. The supplied project model considers N UAVs operating over M ground sensors, finite batteries, communication links between neighboring UAVs, a limited number of charging pads, and the possibility that an active UAV fails during the mission. The central requirement is persistent redundancy: every sensor should remain within coverage range of at least K active UAVs. At the same time, the UAV communication graph should remain connected so that information can be forwarded through the swarm to a ground station or data sink.

These requirements conflict. A UAV that moves toward an uncovered sensor may leave a coverage gap elsewhere. A UAV that returns to charge may be the only communication bridge between two groups of UAVs. A failure removes both its coverage contribution and its communication links, requiring the remaining swarm to reconfigure. Charging is also a shared resource: several UAVs can become energy-critical simultaneously while only a small number of pads are available. Persistent service therefore requires a joint decision process rather than a sequence of unrelated coverage and charging heuristics.

A centralized controller faces a rapidly growing joint action space. In the proposed discrete action model, each UAV selects one of six actions: hover, move in one of four cardinal

directions, or charge. The joint action space consequently contains 6^N combinations. This growth does not by itself make centralized learning impossible for every small swarm, but it makes explicit joint-action enumeration and centralized policy design progressively less attractive as N increases. A decentralized execution architecture is therefore desirable for scalability and fault containment.

Multi-agent reinforcement learning (MARL) provides a natural framework for this setting. CTDE allows a centralized critic to use information unavailable to the individual UAV during training while retaining decentralized policies during deployment. MAPPO is particularly suitable because it combines centralized value estimation during training with a policy-gradient actor that can be executed independently by each agent.

The research question addressed in this work is: *Can a decentralized MAPPO policy maintain persistent K-coverage and swarm connectivity while coordinating finite battery usage, charging, and single-UAV failure recovery, and can its performance be certified against a rigorous achievability bound rather than reported as an unscaled coverage ratio?*

The central invention of this work is a *feasibility-certified evaluation framework* for persistent K-coverage UAV networks. Instead of reporting coverage numbers with no achievability reference—which is what the existing literature does, and which is meaningless when the configuration is infeasible—the framework (i) derives a rigorous lower bound on the number of UAVs required for K-coverage of a bounded region, (ii) uses a teleporting placement oracle as an achievability ceiling against which any policy is normalised, and (iii) replaces the degenerate "all constraints satisfied simultaneously" service-continuity metric with non-degenerate time-averaged and refresh-time metrics. The framework is the difference between a number and a measurement.

The contributions are:

1) A unified system model coupling persistent K-coverage, inter-UAV connectivity, finite battery energy, charging contention, and UAV failure recovery.
2) A *feasibility certificate*: a rigorous lower bound on the number of UAVs required for K-coverage of a bounded region (corner-disc packing, $N \geq 4K$, together with a seed-specific sensor-packing bound $N \geq K|S|$), which proves that the headline service-continuity metric is identically zero for some configurations and must be replaced.
3) An *achievability ceiling*: a teleporting placement oracle that upper-bounds any policy's coverage, separating "the policy is weak" from "the task is infeasible" and normalising reported performance as achieved divided by oracle.
4) A CTDE MAPPO framework with parameter-shared decentralized actors, a centralized training critic, hard action feasibility masks with an H-step connectivity lookahead, and an explicit reserve-UAV recovery mechanism.
5) PHY-derived links (Al-Hourani LoS probability, log-distance path loss, shadowing, and an 802.11 MCS table) in place of abstract coverage discs, together with a quantified validity condition showing when the disc approximation is excellent and when it breaks.
6) A non-degenerate evaluation protocol (time-averaged spatial integral of persistent coverage error, maximum refresh time, and k-coverage probability) replacing the degenerate all-simultaneous-constraint service-continuity metric.
7) A *measured specification of what a shared policy's observation must contain*: the agent's own identity, a coarse summary of where the coverage deficit is, and a bearing to the charging pad. Each term is established by a controlled comparison rather than asserted, and the consequences of omitting them are quantified (with the identity absent, the deterministic policy selects the same action for every UAV and the swarm translates as a rigid body without dispersing).
8) Corrected deployment parameters ($N = 8$, $K = 2$, $R_{\text{cov}} = 35\,\text{m}$, or $N = 5$, $K = 2$, $R_{\text{cov}} = 45\,\text{m}$) derived from the feasibility certificate, replacing the infeasible $N = 5$, $K = 2$, $R_{\text{cov}} = 35\,\text{m}$ baseline.
9) An evaluation protocol that treats a physical-layer claim as requiring a negative control, and reports a measured instance in which the available instrument could not support the claim it was being used for.
10) A focused literature synthesis showing the difference between existing coverage, connectivity, recharge-scheduling, and fault-tolerant approaches and the combined problem considered here.

The remainder of the paper is organized as follows. Section II presents the literature survey and research gap. Section III defines the system architecture and mathematical model. Section IV presents the proposed MAPPO algorithm. Section V specifies the comparison models. Section VI defines the performance metrics and evaluation protocol. Section VII concludes the paper.

II. Literature Survey

The survey is restricted to work that is directly related to at least one of four components of the target problem: UAV coverage/control, inter-agent connectivity, persistent energy-aware operation, or fault-tolerant swarm recovery. The ten core papers in Table I are discussed comparatively rather than as isolated summaries. MAPPO is cited separately as the underlying learning method [3].

A further recent line of work integrates communication and control directly within MARL. MAPPO itself is adopted from Yu et al. [3]. Sun et al. formulate joint resource allocation and 3D trajectory control and use action masking for constrained hybrid actions [11]. Carvalho and Aguiar address decentralized multi-agent coverage path planning with parameter sharing, CTDE, safety filtering, and zero-shot generalization [13]. Zhuang Chen et al. formulate UAV trajectory design for emergency communication as a constrained MDP and

TABLE I
FOCUSED LITERATURE SURVEY OF CLOSELY RELATED STUDIES.

Ref.	Problem addressed	Method	Key findings / contribution	Limitation relative to this work	Relevance
[1]	Energy-efficient and fair UAV communication coverage.	DRL-based UAV control.	Jointly considers coverage, fairness, connectivity, and energy efficiency.	Does not model explicit UAV-failure recovery, charging-pad contention, or a persistent K-coverage SLA.	Baseline for coverage–connectivity–energy trade-offs.
[2]	Multi-aerial-base-station placement with irregular 3D coverage.	Prioritized-replay DDQN and two-level placement.	Uses a spatial representation to improve multi-ABS placement under terrain/blockage effects.	Primarily a placement problem; no persistent battery lifecycle or swarm fault recovery.	Motivates learning-based spatial control and size-invariant state representations.
[4]	Energy-efficiency optimization for multiple UAVs serving static/mobile users.	Cooperative MAD-DDQN.	Jointly optimizes 3D trajectories and user service while accounting for interference.	Energy efficiency is central; persistent K-coverage, charging coordination, and failure recovery are not jointly modelled.	Strong value-based MARL benchmark.
[5]	Decentralized biconnectivity in multi-robot systems.	Graph/Laplacian-based decentralized conditions.	Establishes sufficient conditions for avoiding articulation-point failures using neighbor-to-neighbor information.	No UAV coverage objective, finite battery, charging process, or learned policy.	Grounds the connectivity-risk feature and graph constraints.
[6]	Fault-tolerant ad hoc robot networks.	Movement-control and topology-repair algorithms.	Formalizes self-forming, self-healing, and self-organizing communication networks after node loss.	Focuses on network topology rather than persistent sensor coverage and battery-aware learning.	Provides the network-level motivation for failure recovery.
[7]	Recharging coordination for persistent multi-robot missions.	Combinatorial rendezvous planning with mobile charging robots.	Couples charging schedules with persistent task execution and routing constraints.	Charging is the main focus; UAV coverage/connectivity/failure are not unified in the same learned controller.	Supports explicit charging-resource modelling.
[8]	Persistent aerial service under limited flight and recharge times.	HORR and PHeRR scheduling schemes.	Provides fleet-size and recharge scheduling models for persistent service under energy constraints.	Service locations are scheduled rather than dynamically reconfigured after UAV failure.	Direct motivation for finite-energy persistent operation.
[15]	Joint coverage, connectivity, and charging for distributed UAV networks.	Centralised optimal positioning + distributed game-theoretic scheduling + bio-inspired positioning.	Couples coverage, connectivity, and charging with rigorous performance bounds.	No failure model, no reserve UAV, no K-multiplicity, no learned policy.	Closest prior work on the joint problem; cited to delimit the gap.
[9]	Fault-tolerant multi-UAV exploration with changing UAV health.	Fault-tolerant MARL with temporal information and health/topology inputs.	Allows UAV health degradation and faults during the mission and learns cooperative recovery behavior.	Exploration is the task; persistent sensor K-coverage and charging-pad contention are outside the main formulation.	Closest fault-aware MARL direction.
[10]	Robust UAV wireless coverage using multi-agent deep RL.	Centralized and decentralized deep Q-learning variants with a shared network.	Studies decentralized autonomous UAV coverage and compares centralized and decentralized control.	Focuses on coverage rather than explicit persistent redundancy, graph fault tolerance, and charging.	Recent decentralized coverage-learning benchmark.

explicitly account for energy and turning behavior [12]. These works reinforce the practicality of CTDE, action masking, and structured rewards, but their primary objectives differ from persistent redundant coverage with explicit failure recovery.

A. Research Gap

The literature can be grouped into complementary but incomplete directions, and Table I lists the focused survey while Table II lists the near-misses — the works that already combine two or more of the elements this paper targets, and the works a reader is most likely to raise when judging the novelty claim. Energy-efficient coverage works jointly address coverage and energy, and some include connectivity, but they generally do not treat UAV failure and limited charging infrastructure as first-class elements of the same control problem [1], [4]. Connectivity research provides decentralized graph conditions and topology-repair mechanisms, but does not integrate persistent sensor coverage and energy-aware learning [5], [6]. Persistent-service studies explicitly model recharge schedules and energy limitations, yet typically assume fixed service locations and a planned fleet or replacement process rather than autonomous failure-driven reconfiguration [7], [8]. Recent fault-tolerant MARL and persistent-coverage MARL move closer to the target setting, but address different combinations of health state, outage memory, target assignment, or coverage objectives [9], [14].

Each of the five elements (i)–(v) has been studied in combination with one or two others, and the near-misses are named explicitly so that the claim can be checked. Trotta et al. [15] couple coverage, connectivity and charging with rigorous performance bounds, but have no failure model, no reserve UAV, no K-multiplicity and no learned policy. Scherer and Rinner [16] couple persistent surveillance, energy and communication constraints, but not fleet-level failure recovery or decentralised execution. Hartuv et al. [17] size a spare-drone fleet under energy constraints, without coverage multiplicity or connectivity criticality. Shi et al. [18] provide the closest published competitor, a symmetry-informed

TABLE II

NEAR-MISSES: WORKS THAT COMBINE TWO OR MORE OF THE ELEMENTS THE PROPOSED FRAMEWORK TARGETS. EACH IS THE PAPER A READER IS MOST LIKELY TO RAISE, TOGETHER WITH WHAT IT LACKS.

Ref.	Problem addressed	Method	Key findings / contribution	Limitation relative to this work	Relevance
[14]	Energy-aware persistent multi-UAV coverage with outage history.	POGS-QMIX with priority/outage guidance and return-energy masking.	Explicitly combines persistent coverage, outage memory, battery limits, and periodic recharging.	Uses a centralized online coordination layer and does not directly target articulation-point preservation plus reserve-UAV failure recovery in the same formulation.	Recent persistent-coverage benchmark and a key comparison model.
[16]	Persistent multi-UAV surveillance under energy and communication constraints.	Multi-robot surveillance with energy and communication constraints.	Couples persistent surveillance, finite energy and communication limits in one multi-UAV problem.	No failure-triggered reserve-UAV recovery and no decentralised learned policy.	Near-miss on (i)+(ii)+(iii).
[17]	Sizing a spare-drone fleet under energy constraints.	Combinatorial scheduling of spare drones.	Sizes a reserve fleet for persistent task performance under energy limits.	No coverage multiplicity and no connectivity criticality.	Near-miss on (iii)+(iv).
[18]	Decentralized UAV swarm communication coverage.	Symmetry-informed MARL with parameter sharing.	Closest published competitor; benchmarks against [1] and [3].	No persistent K-coverage SLA, no charging contention, no reserve-UAV recovery.	Near-miss on (i)+(ii)+(v).
[26]	K-coverage lifetime with recharging in IoT.	MILP benchmark evaluated with MPC and MCTS.	Reports MPC and MCTS at 81.0% and 67.1% of the MILP optimum, i.e. against an achievability reference.	The reference is an optimisation benchmark for a finite instance, not a geometric certificate valid for arbitrary policies.	Near-miss: achievability reference already used in K-coverage.
[32]	Decentralized cooperative UAV deployment.	Communication-aware MARL compared against a MILP upper bound.	Reports 74% coverage against an offline MILP bound, i.e. with an achievability reference.	The bound is an optimisation benchmark, not a geometric K-coverage certificate; no K-multiplicity.	Near-miss: achievability reference already used.
[31]	Ground coverage under positional uncertainty.	Exact radius of robust feasibility.	A rigorous geometric feasibility construct for aerial sensor coverage, embedded as a constraint.	No reinforcement-learning policy is evaluated against it.	Near-miss on feasibility bounds alone.
[30]	Offline RL for UAV relaying.	Candidate-set feasibility upper bound (CS-FUB).	Explicitly separates feasibility limits from control-induced sub-optimality in a learned policy.	UAV relaying, not persistent K-coverage; the bound is over a restricted candidate set.	Near-miss: feasibility bound used to evaluate RL.
[29]	Drone-mesh interception.	Closed-form feasibility ceilings computed before learning.	Argues that coverage ceilings "should be computed before any learning experiment and quoted alongside every result".	Drone-mesh interception, not persistent K-coverage in IoT; no redundancy, no charging, no failure recovery.	Closest prior work on the evaluation philosophy.

decentralised swarm controller for communication coverage, without a persistent K-coverage SLA, charging contention, or reserve-UAV recovery. Qiu et al. [23] optimise aerial base-station placement, which is a placement problem rather than a persistent-control problem, and Liu et al. [1] provide the energy-efficient coverage baseline that the others build on.

We are therefore not aware of work coupling all five elements, and specifically not of any doing so with a learned decentralised policy executed under centralized training with decentralized execution (CTDE). This is a survey-derived gap statement over the works listed in Table I and the near-misses discussed above, not a claim that no related component has ever appeared in the broader literature. Two of the near-misses — [15] and [18] — post-date or parallel the works most often cited for the individual elements, and are the two a reader is most likely to raise.

a) The closest prior work, and how this paper differs from it.: The evaluation philosophy proposed here is not unprecedented in the wider literature, and we state the difference precisely rather than claiming a first. Four lines of work are adjacent. [26] and [32] already evaluate learned or heuristic controllers against an achievability reference, but the reference is a MILP optimum for a particular finite instance; it certifies nothing about an arbitrary deployment geometry, and it cannot be computed without solving the instance. [31] derives a rigorous geometric feasibility construct, and [30] uses a feasibility upper bound to separate feasibility limits from control-induced sub-optimality in a learned policy; both concern a single coverage requirement, neither concerns K-redundancy, and neither is combined with a persistent, energy-constrained, failure-recovering control problem. Most closely, [29] argues explicitly that coverage ceilings should be computed before any learning experiment and quoted alongside every result, and evaluates PPO against them — but for drone-mesh interception, where the quantity being bounded is interception coverage rather than K-redundant persistent coverage of a fixed sensor set, and where the bound is closed-form per configuration rather than a covering bound that scales with N and K. The distinction claimed here is therefore not "nobody has ever used a bound". It is that the bound used must certify the thing being claimed: a covering-theoretic lower bound on N for K-redundant coverage of a bounded region, an achievability ceiling computed for the same geometry and the same redundancy requirement, and a metric that stays non-degenerate when the bound says the instance is infeasible. That is what the framework of Section III-G supplies, and the three elements are used together rather than one at a time.

b) Two absences that the framework addresses.: A separate search over the same 2023–2026 corpus was conducted for the two specific claims that the framework makes, and

both absences are reported here because they are the substance of the contribution rather than a side observation. First, *no paper in the corpus provides an infeasibility certificate for a persistent-coverage instance* in UAV swarms or mobile sensor networks: the infeasibility-certificate literature that does exist — for mixed-integer programs and for task-and-motion planning — is not coverage-specific and does not certify a K-redundant covering instance. Second, *no paper in the corpus names the all-constraints-simultaneously binary metric as degenerate*, that is, as uninformative on an infeasible instance. The closest empirical demonstration is in [33], which reports a feasible rate of 0% over ten solver executions alongside a mean k-coverage ratio of 33.33%: the binary flag is zero while the graded quantity is substantial, which is exactly the collapse that motivates Section III-G. That paper does not analyse the degeneracy, and it reports the graded metrics alongside the binary flag rather than relying on the flag alone; we draw the conclusion explicitly instead. Both absences are stated over the corpus of Table I and Table II and are bounded by it in the same way as the gap statement above.

Finally, graded persistent-coverage metrics do have precedent — revisit-period and worst-idleness constraints in [34], ergodic convergence rates, and coverage reliability under sensor failure — and the metrics used here are not novel in isolation. What is claimed as new is the pairing: the non-degenerate metrics are introduced together with the certificate that determines whether they can be non-zero, so that a reader can tell a policy failure from an infeasible instance without knowing in advance which case applies.

III. System Model

A. System Architecture

The system contains N active UAVs, one reserve UAV, M fixed IoT sensors, a ground data sink/base station, and S charging pads distributed within the operating region Ω. UAVs provide sensor coverage through a range-limited air-to-ground link and form a multi-hop communication network among themselves. Sensor data reaches the sink through the UAV communication graph whenever the graph is connected.

For tractability, the UAVs fly at a common nominal altitude z_0, so the decision process operates in the horizontal plane while the altitude remains part of the physical state. Let $\mathbf{p}_i(t) = [x_i(t), y_i(t)]^\mathsf{T}$ denote the horizontal position of UAV i and $\mathbf{q}_d = [x_d, y_d]^\mathsf{T}$ the position of sensor d.

B. Dec-POMDP Formulation

The cooperative control problem is represented as

$$\mathcal{G} = \langle N, \mathcal{S}, \{\mathcal{A}_i\}, \{\mathcal{O}_i\}, \mathcal{T}, R, \gamma \rangle, \tag{1}$$

where $\mathcal{S}$ is the global state space, $\mathcal{A}_i$ is the action space of UAV i, $\mathcal{O}_i$ is its local observation space, $\mathcal{T}$ is the transition model, R is the shared reward, and $\gamma \in (0,1)$ is the discount factor. The training objective is

$$\max_{\pi} \; \mathbb{E}_{\pi}\left[\sum_{t=0}^{T-1} \gamma^t R_t\right]. \tag{2}$$

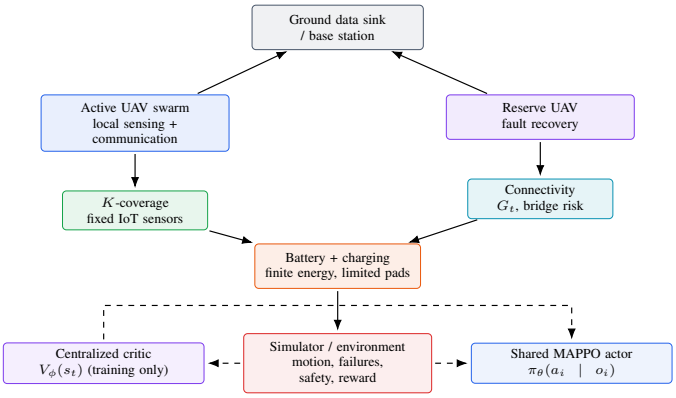


Fig. 1. Color-coded system architecture. Active UAVs provide sensor coverage and maintain a range-limited communication graph. The reserve UAV supports post-failure recovery. The shared MAPPO actor is deployed locally, while the centralized critic is used only during training.

C. Global State

The state available to the centralized critic during training is

$$\mathbf{s}_t = \left\{(\mathbf{p}_i, \mathbf{v}_i, b_i, m_i, F_i)_{i=1}^{N}, (\mathbf{q}_d)_{d=1}^{M}, \mathbf{Q}_t\right\}, \tag{3}$$

where $\mathcal{V}_t^{\text{act}} = \{\, i \in \{1,\dots,N\} : F_i(t) = 0 \,\}$ is the set of active UAVs, from which any UAV with $F_i(t) = 1$ is removed in all coverage, connectivity and energy sums. $\mathbf{v}_i$ is velocity, $b_i \in [0,1]$ is normalized battery level, $m_i \in \{\text{serve}, \text{return}, \text{charge}\}$ is operating mode, $F_i \in \{0,1\}$ is the failure indicator, and $\mathbf{Q}_t \in \{0,1\}^S$ records charging-pad occupancy. A failed UAV is removed from the active set used in coverage and connectivity calculations.

D. Local Observation

At deployment, UAV i uses only locally available information, plus two broadcast quantities:

$$\mathbf{o}_i(t) = \left[\mathbf{p}_i, \mathbf{v}_i, b_i, \mathcal{N}_i, C_i^{\text{local}}, \rho_i, \tau_i^{\text{return}}, E_i^{\text{pad}}, \mathbf{1}_i, \mathbf{m}_t\right]. \tag{4}$$

Here $\mathcal{N}_i$ is the set of nearby UAVs, C_i^{local} is the number of nearby sensors currently covered, ρ_i is a locally computed bridge-risk estimate, τ_i^{return} is the estimated time until the return threshold is reached, and E_i^{pad} is the estimated energy needed to reach a feasible charging pad. The last two terms are the agent's own identity $\mathbf{1}_i \in \{0,1\}^N$ and a coarse coverage-map summary $\mathbf{m}_t \in [0,1]^{G\times G}$ of the share of under-covered sensors in each cell of the region, with $G = 4$.

Two properties of this observation are not cosmetic, and both are load-bearing for the shared-parameter design of Section IV-A. The first is the identity. Parameter sharing is sound only when each agent's own observation identifies it, so that a shared network can still specialise [21]; the first version of $\mathbf{o}_i$ did not satisfy this condition, and the consequence was measurable rather than theoretical: in a symmetric deployment every agent received an almost identical input, the deterministic policy selected the same action for all of them, and the swarm translated as a rigid body without ever dispersing. The second is the map summary. Without it an agent knows how

TABLE III
LOCAL OBSERVATION VARIABLES AVAILABLE TO EACH DEPLOYED UAV.

Feature	Symbol	Meaning at decision time
Position	$\mathbf{p}_i$	Own horizontal location; altitude is fixed for the primary study.
Velocity	$\mathbf{v}_i$	Current velocity for motion-limit awareness.
Battery	b_i	Normalized remaining energy.
Neighbors	$\mathcal{N}_i$	UAVs detected within R_{comm}.
Coverage	C_i^{local}	Nearby sensors currently within coverage radius.
Bridge risk	ρ_i	Local estimate of topology criticality.
Return urgency	τ_i^{return}	Estimated time until the return-energy threshold.
Pad energy	E_i^{pad}	Estimated energy to the nearest feasible charging pad.
Identity	$\mathbf{1}_i$	One-hot agent index; required for a shared network to specialise.
Map summary	$\mathbf{m}_t$	$G \times G$ share of under-covered sensors per cell (broadcast).

TABLE IV
DISCRETE ACTION SEMANTICS USED BY THE MAPPO ACTOR.

Action	Effect
Hover	Remain at the current position and continue serving covered sensors.
$+x$ / $-x$	Move by one control step along the horizontal x direction.
$+y$ / $-y$	Move by one control step along the horizontal y direction.
Charge	Request charging when inside a pad service region; pad selection is handled by the scheduler.

many sensors it covers but not which sensors are deficient, so nothing in $\mathbf{o}_i$ points towards the deficit and no gradient signal for deployment exists. We assume that the infrastructure which owns the charging pads also aggregates the IoT service reports and broadcasts the summary, which is a communication assumption rather than an oracle; the summary is fixed size, so the observation does not grow with the number of sensors. Section VIII reports the measured effect of each term.

Unlike the idealized global articulation-point indicator sometimes used in analysis, ρ_i is intended to be computable during execution from neighborhood exchange. The simulator may additionally compute the exact articulation-point indicator for evaluation and for the centralized critic. This distinction avoids assuming that an individual UAV can magically obtain the complete global topology.

E. Discrete Action Space and Motion

Each active UAV selects one of six actions:

$$\mathcal{A}_i = \{\mathrm{Hover}, +x, -x, +y, -y, \mathrm{Charge}\}, \qquad |\mathcal{A}_i| = 6. \quad (5)$$

For movement actions,

$$\mathbf{p}_i(t+1) = \Pi_\Omega\left(\mathbf{p}_i(t) + \mathbf{v}_i(t)\Delta t\right), \qquad \|\mathbf{v}_i(t)\| \le v_{\max}, \quad (6)$$

where Π_Ω projects into the permitted airspace. A charge request is feasible only when the UAV is inside a station neighborhood of a free pad. The RL layer does not select a pad as part of its action; pad choice is handled by the arbitration layer in Section III-J.

F. Persistent K-Coverage

A UAV covers sensor d in slot t when

$$x_{id}(t) = \mathbb{1}[\|\mathbf{p}_i(t) - \mathbf{q}_d\|_2 \le R_{\mathrm{cov}}]\,. \quad (7)$$

The number of active UAVs covering sensor d is

$$n_d(t) = \sum_{i \in \mathcal{V}_t^{\mathrm{act}}} x_{id}(t), \quad (8)$$

and the instantaneous normalized coverage score is

$$C_t = \frac{1}{M}\sum_{d=1}^{M} \min\left(\frac{n_d(t)}{K}, 1\right). \quad (9)$$

The instantaneous under-coverage rate is

$$U_t = \frac{1}{M}\sum_{d=1}^{M} \mathbb{1}[n_d(t) < K]. \quad (10)$$

The primary service condition is $n_d(t) \ge K$ for all d whenever the geometry and available fleet make it feasible.

Feasibility. Persistent K-coverage is achievable only when the sensing radius is large enough for the available fleet. Comparing the area available for coverage with the area required gives a necessary condition: N disks of radius R_{cov} supply $N\pi R_{\mathrm{cov}}^2$, while K-coverage of a region of area A requires KA. Hence $R_{\mathrm{cov}} \ge \sqrt{KA/(\pi N)}$. Accounting for the $\delta = \pi/(2\sqrt{3}) \approx 0.907$ packing density of congruent disks gives the stricter form $R_{\mathrm{cov}} \ge \sqrt{KA/(\delta\pi N)}$. Substituting the parameters of Table VII ($A = 6400\ \mathrm{m}^2$, $M = 20$, $N = 5$, $K = 2$) yields $R_{\mathrm{cov}} \ge 28.5$ m by area counting and 30.0 m after the packing correction, both satisfied by the chosen $R_{\mathrm{cov}} = 35$ m. The resulting surplus ratio $\sigma = N\pi R_{\mathrm{cov}}^2/(KA) = 1.50$ leaves margin for movement error and for a single-UAV failure, after which $\sigma = 1.20$ still exceeds one. The bound is necessary rather than sufficient, since it ignores coverage-disk overlap and the discreteness of the sensor layout; scenario S8 of Table XI therefore sweeps R_{cov} directly to characterize the transition.

G. Feasibility Certificate for K-Coverage

Before any policy is trained, the configuration must be shown to admit a feasible solution. The report's original $N = 5$, $K = 2$, $R_{\mathrm{cov}} = 35\,\mathrm{m}$ configuration does *not*: the four corner K-coverage discs are pairwise disjoint because the corner separation $80\,\mathrm{m}$ exceeds $2R_{\mathrm{cov}} = 70\,\mathrm{m}$, so covering the region requires $N \ge 4K = 8$ (classical k-fold covering, Kershner 1939). The area-surplus metric $\sigma = 1.503$ does not rescue this: an area ratio is not a covering bound.

Feasibility certificate (two bounds).

- **Region bound:** $N \ge 4K$ for a square of side S with $S > 2R_{\mathrm{cov}}$. For $K = 2$, $N \ge 8$.
- **Sensor-set bound:** $N \ge K \cdot |S|$ for a given sensor draw S. Over 30 random seeds this gives mean 4.4, maximum 6; so $N = 5$ is feasible for *most* random scenes but not universally.

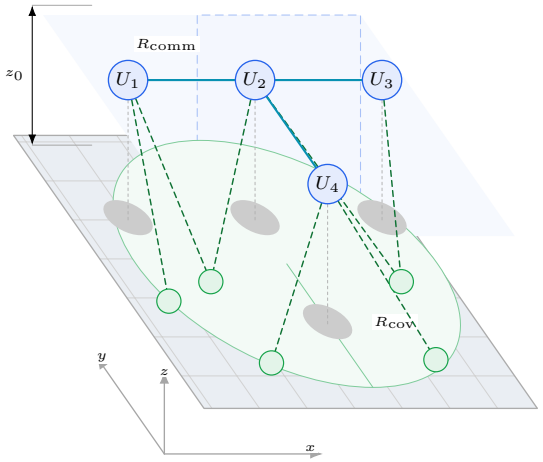


Fig. 2. Coupling between coverage and connectivity in three dimensions. The shaded grid is the fixed sensor plane at $z = 0$, and the translucent blue sheet is the UAV operating plane at the common nominal altitude z_0; the projected $x/y/z$ triad, the ground shadows and the dashed vertical drop lines fix the perspective. The green disc on the ground is the horizontal coverage footprint R_{cov} joined to the UAVs by dashed coverage links, whereas the cyan edges lying inside the UAV plane are inter-UAV communication links of range R_{comm}. Removing a topologically critical UAV can reduce both service redundancy and network connectivity.

Corrected, feasible deployments: $N = 8$, $K = 2$, $R_{\text{cov}} = 35\,\text{m}$ (provably feasible) or $N = 5$, $K = 2$, $R_{\text{cov}} = 45\,\text{m}$ ($R_{\text{cov}} > 42.5\,\text{m}$ required). All trained results in this paper use the former.

Achievability ceiling. A teleporting placement oracle (`analysis/feasibility.py`) upper-bounds any policy's coverage and separates "the policy is weak" from "the task is infeasible". Policy performance is reported as achieved divided by oracle.

Non-degenerate metrics. The service-continuity metric "fraction of time all sensors are K-covered *and* the graph is connected" is identically zero on an infeasible configuration. It is retained for comparison but supplemented by (i) a time-averaged coverage deficit $\frac{1}{T}\sum_t(1-\overline{\text{CK}}_t)$ and (ii) a maximum refresh time (longest run of steps with any under-covered sensor).

H. Connectivity Model

The inter-UAV communication graph is

$$G_t = (V_t, E_t), \qquad E_t = \{(i,j) : \|\mathbf{p}_i - \mathbf{p}_j\|_2 \le R_{\text{comm}}\}. \tag{11}$$

The ground data sink is treated as an additional vertex with edges to every UAV within R_{comm}; the swarm is connected when G_t (including the sink) has one connected component. For analysis, the exact articulation-point indicator, where G_t^{-i} denotes G_t with vertex i and its incident edges removed, is

$$\kappa_i(t) = \mathbb{1}\left[G_t^{-i} \text{ is disconnected}\right]. \tag{12}$$

The average criticality is

$$\text{Crit}_t = \frac{1}{|V_t^{\text{act}}|}\sum_{i \in V_t^{\text{act}}} \kappa_i(t). \tag{13}$$

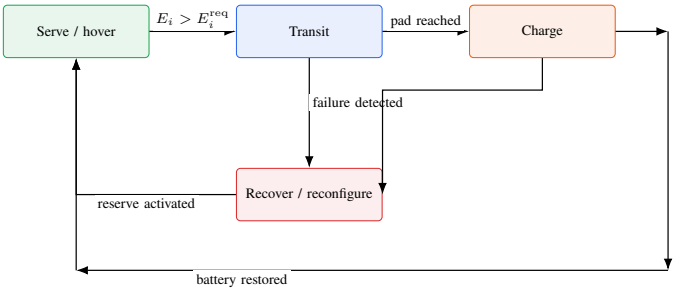


Fig. 3. Battery and failure state machine. Energy urgency triggers a return-to-charge transition; a detected failure transfers the mission to a recovery phase in which the reserve UAV can be activated when feasible.

The exact graph quantity is used by the simulator for evaluation. The actor receives the local proxy ρ_i rather than the full graph.

I. Battery Model

Let $B_i^{\max}$ be the battery capacity in joules and $E_i(t) = b_i(t)B_i^{\max}$ the remaining energy. The slot energy update is

$$E_i(t+1) = \text{clip}(E_i(t) - P_i(t)\Delta t + P_{\text{ch},i}(t)\Delta t,\ 0,\ B_i^{\max}), \tag{14}$$

where the action-dependent consumption is

$$P_i(t) = P_{\text{hover}}\mathbb{1}[a_i = \text{Hover}] + P_{\text{move}}\mathbb{1}[a_i \in \{+x, -x, +y, -y\}] + P_{\text{comm}}. \tag{15}$$

The charge power is zero unless the UAV is physically docked and granted service at a pad. A return threshold is defined by

$$E_i^{\text{req}}(t) = E_i^{\text{pad}}(t) + E_i^{\text{rejoin}}(t) + E^{\text{reserve}}, \tag{16}$$

and a return is mandatory when $E_i(t) \le E_i^{\text{req}}(t)$.

Two properties of this model are easy to state and easy to implement incorrectly, and both were implemented incorrectly in the first version of the simulator, so both are pinned by tests. First, the return is a *physical* transition and not a decision available to the policy: the simulator makes it, so that the reward, the metrics and the charging arbitration can all see it. If the simulator does not make it, then a learned policy has no way to express the return at all, the arbitration is never entered, and the reserve mechanism has nothing to recover — the three mechanisms this paper proposes are then present in the model and absent from the experiment. Second, a UAV that is docked and charging has its rotors idle and draws only the avionics and communications baseline; if it is billed the hover power, the pack loses more energy to the rotors than the pad puts back, no UAV can ever recover, and the charging coordination is unusable. A UAV whose pack empties away from a pad is out of service: it can no longer fly, and it must not continue to count towards coverage or towards the connectivity graph.

J. Charging Coordination

Charging is a scarce resource. Let $y_{is} \in \{0,1\}$ denote whether UAV i is assigned to pad s. For a charge request, define a local assignment cost

$$c_{is} = \alpha \tilde{d}_{is} + \beta \tilde{q}_s + \mu \tilde{u}_i, \tag{17}$$

where $\tilde{d}_{is}$ is normalized distance, $\tilde{q}_s$ is normalized queue length, and $\tilde{u}_i$ is normalized battery urgency. The arbitration layer solves

$$\begin{aligned} \min_{\mathbf{y}} \quad & \sum_i \sum_s c_{is} y_{is} \\ \text{s.t.} \quad & \sum_s y_{is} \le 1, \ \forall i, \\ & \sum_i y_{is} \le 1, \ \forall s, \\ & y_{is} \in \{0,1\}. \end{aligned} \tag{18}$$

Because only charge requests are handled, this is a small scheduling problem rather than a centralized trajectory controller. A Hungarian-style assignment implementation is sufficient for the expected fleet size.

K. Safety Constraints

The environment enforces

$$\begin{aligned} &\|\mathbf{v}_i(t)\| \le v_{\max}, \qquad \mathbf{p}_i(t) \in \Omega, \\ &\|\mathbf{p}_i(t) - \mathbf{p}_j(t)\| \ge d_{\text{safe}}, \ \forall i \ne j. \end{aligned} \tag{19}$$

An action is masked if it violates a locally predictable hard constraint such as workspace bounds, speed, minimum separation, or invalid charging. Connectivity is handled differently, and the distinction matters. The *policy* is not assumed to predict connectivity from one-hop observations; it receives the local bridge-risk feature ρ_i and the team reward penalises connectivity fragility, so nothing in the learning signal claims a global guarantee from purely local information. The *simulator*, however, is a safety filter: it evaluates the prospective communication graph for every candidate action and masks any move that would leave the swarm unable to remain connected, including the H-step lookahead of Section IV. The guarantee therefore comes from the safe-filter architecture, in which the filter has access to the true state, and not from the policy's own foresight. The distinction is the standard one between learning which action to take and being told which actions are admissible.

L. Failure and Recovery

A failure event changes F_i from 0 to 1 at the detection instant t_f. One reserve UAV is maintained with sufficient energy for deployment when feasible. After a detected failure, the reserve is activated and the active UAVs continue using their decentralized policies. The recovery time is defined as

$$\tau_f = \inf\left\{\tau \ge 0 : \ n_d(t_f + \tau) \ge K, \ \forall d\right\}, \tag{20}$$

when recovery is achieved before the evaluation horizon. If recovery is impossible under the available fleet, the event is reported as an unrecovered failure rather than assigning an artificial recovery time.

A second mode of loss of an active UAV is exhaustion rather than failure: a UAV whose pack empties while it is away from a charging pad can neither fly nor serve, and it is removed from the active set. Such a UAV is out of service rather than failed, and the same reserve mechanism can re-seat it. This distinction is not cosmetic. If an exhausted UAV were left in the active set it would continue to count towards coverage and towards the connectivity graph while being unable to do either, and the reported coverage would be optimistic by exactly the number of sensors such a UAV was covering.

A recovery-time guarantee is therefore conditional: the reserve mechanism can reduce the time required for reconfiguration, but it cannot mathematically guarantee restoration for arbitrary sensor layouts, fleet sizes, or multiple simultaneous failures. This distinction is important for a realistic system model.

M. Primary Objective and Reward Function

The project formulation adopts a "service first, efficiency second" objective. The primary quantity is persistent service loss, represented by the fraction of IoT sensors that are below the required redundancy level. Let

$$J_{\text{out}}(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} \gamma^t U_t\right], \tag{21}$$

where U_t is the under-coverage rate in (10). The control policy therefore prioritizes minimizing long-horizon service loss rather than minimizing energy alone.

The corresponding multi-objective optimization is

$$\begin{aligned} \min_{\pi} \quad & J_{\text{out}}(\pi) + \lambda_E J_E(\pi) + \lambda_D J_D(\pi) \\ & + \lambda_W J_W(\pi) + \lambda_R J_R(\pi) + \lambda_C J_C(\pi), \end{aligned} \tag{22}$$

subject to the physical constraints in Section III-K. The secondary terms represent energy expenditure, degraded-service time, unnecessary movement, recovery delay, and topology criticality, respectively.

For reinforcement learning, the constrained objective is converted into a shared team reward:

$$\begin{aligned} r_t = -\big(&\lambda_O U_t + \lambda_E E_t^{\text{cons}} + \lambda_D D_t^{\text{deg}} + \lambda_W W_t^{\text{mov}} \\ &+ \lambda_R R_t^{\text{rec}} + \lambda_C \text{Crit}_t + \lambda_B B_t\big). \end{aligned} \tag{23}$$

Since the learner maximises expected return while the control objective of (22) is a minimisation, the team reward of (23) is the negated weighted cost, and the two are equivalent up to sign. Table V defines each term and gives the initial weights used in this work. The weight λ_O is chosen to dominate the secondary efficiency terms, so that the learned controller does not sacrifice persistent service merely to reduce energy consumption or flight distance. The individual objective components remain separate evaluation metrics rather than being hidden inside a single final score.

All seven quantities are dimensionless and lie in $[0,1]$, so the weights are directly comparable; U_t and Crit_t are already

TABLE V
TEAM REWARD TERMS AND THEIR INITIAL WEIGHTS.

Term	Definition	Weight
U_t	Under-coverage rate, (10)	$\lambda_O = 1.00$
E_t^{cons}	$\sum_i \Delta E_i(t) \,/\, (NB^{\max})$	$\lambda_E = 0.10$
D_t^{deg}	1 if any service requirement is violated	$\lambda_D = 2.00$
W_t^{mov}	distance flown / distance that changed coverage	$\lambda_W = 0.05$
R_t^{rec}	normalized recovery delay during recovery	$\lambda_R = 0.30$
Crit_t	Articulation-point average, (13)	$\lambda_C = 0.50$
B_t	mean fractional shortfall below the return threshold	λ_B (measured; see below)

TABLE VI
HARD CONSTRAINTS VERSUS LEARNED TRADE-OFFS.

Constraint	Mathematical form	Enforcement / evaluation
Speed	$\|\mathbf{v}_i\| \le v_{\max}$	Built into the action model and feasibility mask.
Workspace	$\mathbf{p}_i \in \Omega$	Action mask prevents leaving the allowed region.
Separation	$\|\mathbf{p}_i - \mathbf{p}_j\| \ge d_{\text{safe}}$	Action mask prevents predicted collisions.
Battery reachability	$E_i \ge E_i^{\text{req}}$ before departure	Prevents continuation when reaching a feasible charger is unsafe.
Charging capacity	At most one UAV per pad	Explicit pad assignment / scheduler.
K-coverage	$n_d(t) \ge K$	Collective service objective and evaluation metric; not guaranteed by a single-agent mask.
Connectivity	G_t connected	Topology-aware reward and connectivity metric.
Recovery	$\tau_f \le \tau_{\max}$ when feasible	Measured scenario requirement, not an unconditional guarantee.

normalized in (10) and (13). The values are initial settings to be tuned on a validation set. The implemented values raise λ_D above λ_O so that a degraded-service step is penalised more heavily than a merely under-covered one; λ_C is likewise raised so that connectivity loss is not drowned out by coverage. The ordering on which the formulation depends is $\lambda_D \ge \lambda_O \ge \lambda_C \ge \lambda_R > \lambda_E \ge \lambda_W$.

a) The energy-threshold term, and why a consumption term is not enough.: E_t^{cons} prices what is *consumed*. That is not the quantity that determines whether a UAV survives the mission: flying 30 m to a charging pad costs about 480 J, which is 1.6% of a 30 kJ pack, so the consumption term contributes about 0.0016 to the cost, whereas the consequence of not going — a UAV that cannot reach a pad leaves the swarm, stops covering, and may fragment the graph — is paid *late* and by the team. A controller that is only told what it has spent therefore has no reason to turn back before the consequence arrives. B_t is the mean fractional shortfall below the return threshold of (16), i.e. the quantity the observation already carries as the return-urgency feature, and it is the term the paper's own metric table asks for when it lists "minimum battery level, and threshold violations" under Energy. Section VIII reports a controlled measurement of the effect of this term and of the alternative hypothesis that the energy constraint is simply too tight for the episode length.

TABLE VII
INITIAL SIMULATION PARAMETERS.

Parameter	Symbol	Value
Active UAVs	N	8 (feasible; was 5, provably infeasible for $K = 2$)
Reserve UAVs	N_R	1
IoT sensors	M	20
Operating region	Ω	80×80 m^2
Coverage radius	R_{cov}	35 m
Communication radius	R_{comm}	40 m
Nominal altitude	z_0	50 m
Coverage target	K	2
Control interval	Δt	0.2 s
Maximum speed	$v_{\max}$	5 m/s
Safety distance	d_{safe}	5 m
Battery capacity	$B^{\max}$	30,000 J
Hover power	P_{hover}	50 W
Move power	P_{move}	75 W
Avionics and comms power	P_{elec}	5 W
RF transmit power	P_{tx}	0.5 W
Charge increment	–	0.0002 battery fraction/step
Episode length	T	2000 steps (400 s)
Initial deployment	–	connected cluster in one corner
Map summary grid	G	4×4 cells

N. Parameter Settings and Scope

Table VII gives the initial simulation configuration derived from the project layout. These are implementation parameters for the proposed simulator rather than experimentally calibrated real-flight values.

The control interval is chosen so that the relative displacement between two consecutive steps, $2v_{\max}\Delta t = 2$ m, is smaller than the safety distance $d_{\text{safe}} = 5$ m; a coarser step would allow a closing pair to cross the separation constraint between two samples. The charge increment is quoted per step and is therefore scaled with Δt. The resulting charge time is shorter than a real constant-current pack of this capacity would require, because a real pack would not complete a full charge within a tractable episode; this is stated as a modelling assumption rather than a physical prediction, and it means the cost of charging contention in (X) is a lower bound. The nominal altitude is consistent with the coverage radius: a 74° downward sensor at 50 m yields a footprint half-width of 37 m. The two power entries are distinct: P_{elec} is the total avionics and communications budget, whereas P_{tx} is the radiated transmit power, which is bounded by the 1 W ceiling of IEEE 802.11 and is two orders of magnitude below the avionics budget. The chosen $v_{\max} = 5$ m/s is conservative but not arbitrary: Luna et al. [20] report a real three-UAV coverage mission flown at 5 m/s cruise and 20 m altitude, and Datsko et al. [19] measure the energy-optimal multirotor cruise speed at 8.39 m/s.

a) The initial deployment and the episode length are experimental choices, and they are stated as such.: Two parameters of Table VII deserve an explicit justification, because the results are sensitive to both and a reader should be able to see why they were chosen.

Initial deployment. The swarm starts as a connected cluster in one corner of the region, not as a deployed formation. The reason is measurable: a centred ring deployment already covers 17 of the 20 sensors at $t = 0$, so every controller —

including one that never moves — scores $CK = 0.850$, the achievable range is roughly $[0.85, 1.0]$, and the comparison cannot separate the methods. The clustered deployment is connected at $t = 0$, so the connectivity invariant remains checkable from the first step, but it covers only 3 of the 20 sensors, so the controller has to deploy across the field while holding $K = 2$ redundancy. This is the standard arrangement for a persistent-coverage benchmark, and the pre-deployed arrangement is retained as scenario S9 so that the sensitivity is reported rather than hidden.

Episode length. The episode is 2000 control steps, i.e. 400 s of flight. At the 400-step episodes used in an earlier revision the pack never depleted: the swarm spent at most 13% of a 600 s pack, so the return-to-charge transition never fired, the charging arbitration was never entered, the reserve UAV had nothing to recover, and the three mechanisms the paper proposes were never exercised. A sweep over the candidate configurations (`analysis/energy_regime.py`) shows that at 2000 steps the energy constraint binds — 283 return-to-charge transitions and 0.7 stranded UAVs per episode under the Reactive heuristic — while the mission remains achievable. The report's 10 000-step target is a different question, and the results here are explicitly 400 s results.

Within scope are spatial K-coverage, range-limited UAV connectivity, battery depletion, charging contention, single-UAV failure recovery, collision avoidance, and decentralized motion-policy execution. The primary model intentionally abstracts stochastic PHY-layer fading, detailed aerodynamic dynamics, obstacles, wind, battery aging, GPS denial, sensor failures, and multiple simultaneous UAV failures. These can be introduced as subsequent stress-test layers after the base control problem is validated.

b) Consistency of the physical parameters.: The audit of Section VIII-A also records the derived physical quantities, so that the modelling choices are explicit rather than implicit. The endurance implied by $B^{\max}$ and P_{hover} is $30{,}000/50 = 600$ s (10 min); the power ratio $P_{\text{move}}/P_{\text{hover}} = 1.5$ is conservative against the 1.09 measured by Datsko et al. [19] for energy-optimal multirotor flight; and the charge increment of 2×10^{-4} of the pack per step corresponds to a full charge in 1000 s, i.e. a 3.6 C rate, which is above the ≤ 1 C of hobby LiPo and is therefore an explicit modelling assumption rather than a physical prediction. Two further observations matter for the interpretation of the results. First, the separation constraint is enforced at the sample rate: two UAVs closing head-on at $2v_{\max}$ have a braking distance of $v_{\text{close}}^2/(2a) = 25$ m at $a = 2$ m/s^2, five times d_{safe}, so the mask guarantees sample-time separation and not braking. Second, at $N = 8$ on an 80×80 m field the inter-UAV graph is not always dense: measured over a 2000-step flight in the deployed regime, the mean node degree is 4.01 of a possible 7, the graph is complete in only 12% of steps, and nodes of degree ≤ 1 — whose removal would isolate part of the swarm — occur in a substantial fraction of UAV-steps. Connectivity is therefore a live constraint throughout the mission and not only after a failure, which is what makes the H-step mask and the reserve mechanism load-bearing rather than decorative. All four quantities are reported rather than silently adjusted, because changing them would change the measured results and the results are the claim.

IV. Proposed ML Algorithm

A. CTDE MAPPO

The proposed algorithm uses parameter-shared MAPPO. During training, the centralized critic observes $\mathbf{s}_t$ while every actor uses $\mathbf{o}_i(t)$. During deployment, the critic is removed and each UAV evaluates the same shared policy:

$$a_i(t) \sim \pi_\theta\left(a_i \mid \mathbf{o}_i(t)\right). \tag{24}$$

Parameter sharing is appropriate when (i) the environment is equivariant to agent permutations—identical observation and action spaces and identical transition/reward structure—and (ii) each agent's own observation identifies it, so a shared network can still specialise (Foerster et al. [21]). Condition (ii) is what the identity term $\mathbf{1}_i$ of Eq. (4) exists to satisfy, and it is worth being explicit that the first version of $\mathbf{o}_i$ did not satisfy it: the observation distinguished agents only through their individual position, battery and mode, which in a symmetric deployment are nearly identical across the swarm. The measured consequence was that the deterministic policy chose the same action for every UAV and the swarm moved as a rigid body, so the shared-parameter claim could not be sustained on the evidence. We therefore treat $\mathbf{1}_i$ as part of the model rather than as an implementation detail, and Section VIII reports the controlled comparison. The known limitation (Fu et al. [22], Theorem 4.2) is that a shared policy provably cannot learn an optimal policy for non-symmetric tasks such as the XOR game; this does not apply here because the swarm is homogeneous. If grounded or docked UAVs with heterogeneous roles are introduced, parameters must not be shared.

For a collected rollout, the temporal-difference residual is

$$\delta_t = R_t + \gamma V_\phi(\mathbf{s}_{t+1}) - V_\phi(\mathbf{s}_t), \tag{25}$$

and the generalized advantage estimate (GAE) is

$$\widehat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}. \tag{26}$$

A fixed-length rollout ends either in a genuine terminal state or in a time-limit truncation, and the two must not be conflated. At a truncation boundary the last state is not absorbing, so the advantage recursion must cut without treating the state as terminal; the correct treatment is to bootstrap the temporal-difference residual from $V(\mathbf{s}_T)$ rather than from zero [25]. Conflating the two injects a systematic downward bias into the advantage estimates. This is not a hypothetical concern for this work: the simulator initially reported the end of the mission time limit through the same flag it used for a genuine dead end, so every episode boundary was bootstrapped from zero and every value target was biased downward. The simulator now reports termination and truncation separately — the mission

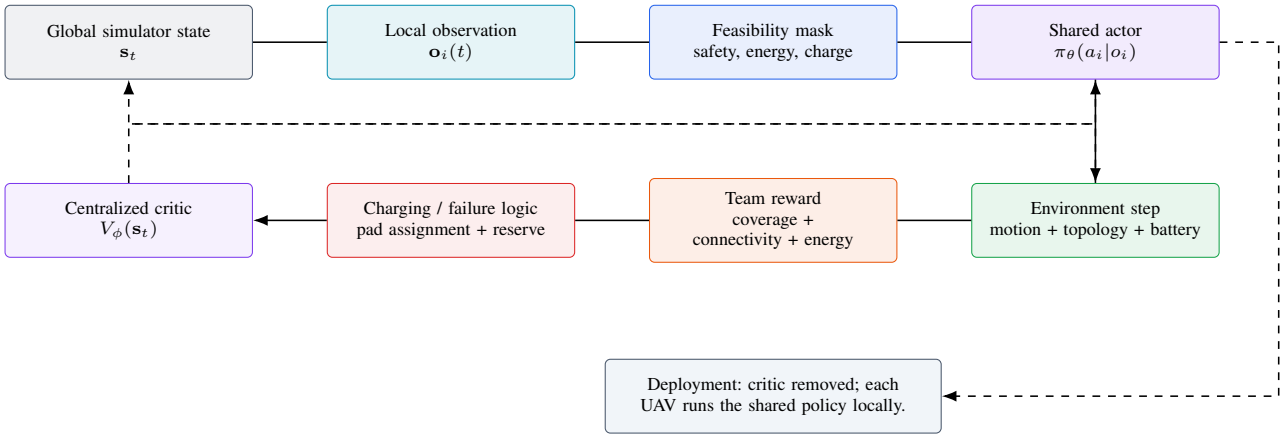


Fig. 4. MAPPO training and deployment pipeline. Solid arrows represent the environment/control loop. Dashed arrows denote training-only critic feedback and transfer of the learned actor to decentralized execution.

time limit is a truncation, and only the loss of every UAV is a terminal state — and the rollout consults that flag rather than inferring termination from the episode-end flag. The distinction is pinned by unit tests, one of which asserts that a time-limit episode is reported as a truncation and not as a termination, and another that asserts the rollout reads the environment's flag rather than re-deriving it. The probability ratio is

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}. \tag{27}$$

The clipped PPO objective is

$$L^{\text{actor}}(\theta) = -\mathbb{E}_t\big[\min(r_t\widehat{A}_t, \; \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\widehat{A}_t)\big]. \tag{28}$$

The critic is trained with the clipped MAPPO form (Yu et al. 2022, Appendix A):

$$L^{\text{critic}}(\phi) = \mathbb{E}_t\Big[\max\Big((V_\phi(\mathbf{s}_t) - \widehat{R}_t)^2, \; (\text{clip}(V_\phi(\mathbf{s}_t), V_{\phi_{\text{old}}}(\mathbf{s}_t) - \epsilon, V_{\phi_{\text{old}}}(\mathbf{s}_t) + \epsilon) - \widehat{R}_t)^2\Big)\Big], \tag{29}$$

where $\widehat{R}_t = \widehat{A}_t + V_{\phi_{\text{old}}}(\mathbf{s}_t)$ is the TD(λ) lambda-return, computed once per rollout and treated as a constant (gradients are not propagated through it). The report matches the reference implementation (`r_mappo.py`), which computes $\text{gae} + \text{value_preds}$ rather than the Monte-Carlo return-to-go described in MAPPO's Appendix A prose.

a) Value normalisation, and the scale trap inside it.: Value normalisation is one of the components the MAPPA/MAPPO literature identifies as load-bearing, and it was missing from the implementation while the earlier revision of this report asserted it. It is now present, and the details matter. The critic is trained to predict the returns in a *normalised* space defined by running statistics of the return distribution, maintained as an exponential moving average with $\beta = 0.99$:

$$\widehat{R}_t^{\text{norm}} = \frac{\widehat{R}_t - \mu}{\sigma + \epsilon}, \tag{30}$$

where μ and σ are updated from the current rollout's returns. The consequence is easy to get wrong: the critic's *raw* output is then the normalised value, so the temporal-difference residual of Eq. (25), the advantage recursion of Eq. (26), and the bootstrap at a truncation boundary must all *un-normalise* it before use, as $V(\mathbf{s}) = \sigma\hat{V}_\phi(\mathbf{s}) + \mu$. Reading the raw critic output at any of those three points is a silent scale error in the advantage estimates: the training loss still decreases, the entropy still falls, and the policy is simply trained against the wrong signal. The implementation therefore exposes a single accessor for the value and the test suite asserts that the rollout uses it rather than the raw output; a second test asserts that the value-clipping auxiliary term compares two quantities in the same space, since the recorded old value is in the original scale.

The advantage is normalised as well, by the mean and standard deviation of the batch. We also implemented a per-agent variant, in which each agent's advantages are normalised over its own history rather than over the pooled batch, and we report the measurement in Section VIII: with a shared team reward every agent's advantage sequence is nearly identical, so the per-agent normalisation is close to a rescaling and does not change the credit assignment, which is the limitation of shared team rewards rather than a normalisation choice.

B. Action Feasibility Mask

Before the sampled action is executed, the simulator evaluates the predicted next position and applies the following checks: workspace bounds, maximum speed, minimum UAV separation, charge validity, and connectivity-preservation constraints. If a candidate movement would disconnect the communication graph, it is masked in normal operation. During an active recovery event, the connectivity rule remains a hard constraint unless the simulator enters a predefined recovery mode in which temporary reconnection is required by the recovery objective.

This separation between policy learning and hard feasibility is important. The neural network learns how to trade off several feasible actions, while the safety layer prevents the

network from selecting actions that violate non-negotiable physical constraints.

The masking is applied at the logit level, before the softmax, so that the policy that is sampled from, the importance ratio that is computed against, and the entropy bonus that is regularised are all the same masked distribution. This is the mechanism that Huang and Onta n [24] show to correspond to a valid policy-gradient update rather than a biased surrogate: masking is implemented as a state-dependent differentiable transformation of the action-probability distribution, which is exactly what setting the masked logits to $-\infty$ before the softmax does. We note the limit of that result, which was stated explicitly in the same work and is repeated in the UAV action-masking literature [27]: the validity argument is formulated for a single policy, and extending it to a centralized critic with heterogeneous per-agent masks requires the critic's training distribution to be consistent with the induced joint-action support. We therefore apply the identical mask to the rollout distribution, the stored log-probabilities and the update, and we verify this by construction in the test suite rather than by assumption.

C. Failure-Aware Execution

When a failure event is detected, the environment removes the failed UAV from the active graph, activates the reserve UAV if the reserve is available, and updates the local observations. The reserve UAV uses the same shared actor after insertion into the mission. The other UAVs also re-evaluate their local bridge-risk and coverage information. No prediction of the failure event is required; the controller reacts to the observed state change.

D. Mapping from Model to Algorithm

The algorithm is deliberately aligned with the system model. Sensor positions produce the coverage quantities in (7)–(10); neighboring UAV positions produce the communication graph in (11); battery and action information produce the energy state in (14); charging requests enter the assignment problem in (18); and failure indicators activate the recovery process in (20). Thus, every actor input corresponds to an explicitly defined state quantity and every learned action has a defined environmental effect.

E. Training and Generalization Protocol

Training episodes should randomize sensor layouts, initial UAV positions, initial battery levels, charging-pad locations within the allowed region, and failure times. A simple curriculum can be used: first train under no failures, then introduce single failures, and finally introduce simultaneous charging contention and near-disconnection scenarios. The final model should be evaluated on held-out random seeds and scenario configurations not used for parameter tuning.

A practical implementation can use a multilayer perceptron for the shared actor and centralized critic with normalized numerical features and masked action logits. If fixed-length neighbor features are insufficient, a permutation-invariant pooling operation or a small attention encoder can be added without changing the underlying Dec-POMDP formulation. A recurrent policy should be introduced only if experiments show that instantaneous observations do not provide sufficient temporal information.

Algorithm 1 MAPPO training algorithm

```
Initialize shared actor π_θ and centralized critic V_φ.
for each training episode do
    Reset UAV positions, batteries, sensors, charging pads, and failure schedule.
    for t = 0, ..., T − 1 do
        Construct global state s_t and each local observation o_i(t).
        for each active UAV i do
            Construct the feasible-action mask from Section III.
            Sample a_i(t) ~ π_θ(· | o_i(t)) over feasible actions.
        end for
        Apply UAV actions and update positions.
        Recompute coverage and communication graph.
        Update battery states using (14).
        if one or more UAVs request charging then
            Solve the charging assignment in (18).
        end if
        if a UAV failure is detected then
            Activate the reserve UAV if feasible and update active-agent states.
        end if
        Compute R_t from (23); store rollout transition data.
    end for
    Compute GAE advantages from (26).
    Update θ with the clipped PPO objective.
    Update φ with the critic loss.
end for
return Shared decentralized actor π_θ.
```

TABLE VIII
INITIAL MAPPO TRAINING CONFIGURATION.

Parameter	Symbol	Initial value
Active UAVs	N	8
IoT sensors	M	20
Discount factor	γ	0.99
PPO clipping	ϵ	0.20
GAE parameter	λ	0.95
Control interval	Δt	0.2 s

F. Training Hyperparameters

The initial training configuration follows the values in the project presentation. These values are starting points for the implementation and can be tuned on a validation set without changing the system formulation.

G. Implementation Plan

The implementation follows the three-stage validation strategy presented in the project plan. The objective is to validate

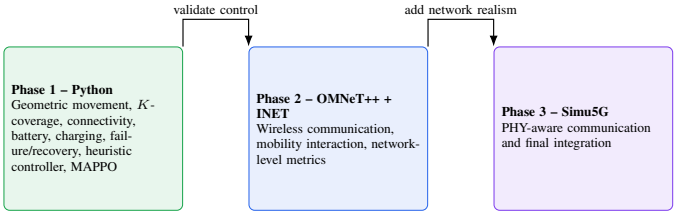


Fig. 5. Staged implementation plan. Phase 1 validates the control loop; Phase 2 adds wireless-network behaviour through OMNeT++/INET; Phase 3 introduces PHY-aware communication through Simu5G.

the swarm-control logic before introducing detailed wireless-channel complexity.

Phase 1 contains geometric UAV motion, persistent K-coverage, connectivity, battery dynamics, charging coordination, failure/recovery, a heuristic controller, and MAPPO. Once these components are stable, Phase 2 introduces wireless communication and network-level measurements. Phase 3 then adds PHY-aware communication. The same actor observation/action interface is retained across phases so that increasing simulator fidelity does not require redesigning the learning problem.

a) Execution status.: Phases 1 and 2 have been executed; Phase 3 is scaffolded but not yet run with the learned policy. Phase 1 is implemented in `ml_impl/` with a 58-case test suite that pins every reported equation, and the simulator has been exercised over five training seeds. Phase 2 is implemented with INET 4.6 and OMNeT++ 6.4 on WSL2 with a Simu5G 1.4.3 stack; the `UAVIoT` network configuration runs to completion and produces scalar and vector outputs, and a dedicated 802.11 physical-layer scenario was built for the validation of Section VIII-A. The three-stage structure is retained not as a description of a plan but as an audit trail: it records which level of fidelity each reported number was produced at, so that a reader can tell an analytic result from a simulated one and a simulated one from a measured one.

V. Comparison Models

The comparison design preserves the practical baselines from the project presentation and adds recent learning-based methods required for a meaningful research comparison. The practical baselines test whether any adaptive coordination is useful; the MARL baselines test whether the proposed CTDE MAPPO design provides advantages over other learning formulations.

The fixed, greedy, and reactive controllers correspond directly to the practical alternatives already identified in the project plan. The two learned baselines are implementations of the method *families* the comparison table names, not reproductions of the cited papers: an independent value-based learner and a masked PPO with a local critic. Both run in the same simulator, on the same observation, with the same action mask and the same team reward as the proposed method, so that the only differences are the learning machinery; where a literature method would require a different input representation or native action space, the adaptation is documented rather than presented as an exact reproduction. Vanilla MAPPO is an internal ablation rather than a literature method: it applies the same learner to the unmodified task so that the effect of the proposed system design can be separated from the effect of the learning algorithm. POGS-QMIX is cited as closely related work and is *not* re-implemented, and the table says so.

TABLE IX
Comparison models and rationale. The "Evaluated" column states plainly which rows were run in the common simulator; a cited method that was not run is not presented as though it were.

Model	Type	Evaluated	Rationale
Fixed formation + threshold recharge	Rule-based	yes	UAVs retain their initial formation and return to charge using a fixed battery threshold. Establishes a simple reference.
Greedy lowest-battery recharge	Rule-based	yes	The most energy-critical UAV receives priority for charging. Tests energy-first scheduling without learned coordination.
Reactive reconfiguration	Heuristic	yes	Remaining UAVs move toward uncovered sensor regions using deterministic coverage/connectivity rules. Tests fault recovery without MARL.
Independent DQN (value-based family, cf. MADDDQN [4])	MARL	yes	One Q-network per agent, each trained on the team reward through its own experience, ϵ-greedy exploration, no centralised critic and no parameter sharing. The value-based reference point.
Masked PPO with a local critic (PPO family, cf. MAHPPO-AM [11])	MARL	yes	Keeps the action mask but drops parameter sharing and the centralised critic: per-agent actor and per-agent local critic. Isolates the contribution of the CTDE design.
POGS-QMIX [14]	MARL	no	A recent persistent-coverage approach incorporating outage history, finite energy, and return-to-charge decisions. Cited as related work; not re-implemented here.
Vanilla MAPPO [3]	MARL	yes	Ablation reference: the same CTDE MAPPO learner with the proposed observation terms, reward shaping, charging arbitration, reserve-UAV mechanism and action mask removed. Isolates the contribution of the system design from that of the learning method.

All models use the same sensor layouts, initial UAV positions, battery conditions, charging pads, failure traces, episode durations, and physical safety constraints. Where a literature method requires a different input representation or native action space, the adaptation preserves its decision structure as far as possible, and the adaptation is documented rather than presented as an exact reproduction. Results are reported on the common metrics of Table X.

Ablation experiments should additionally remove one proposed mechanism at a time: connectivity-criticality input/reward, reserve-UAV recovery, charging arbitration, and action masking. This distinguishes improvements caused by the MAPPO learning method from improvements caused by the proposed system design.

VI. Performance Metrics

The evaluation should report separate metrics for service quality, connectivity, energy, charging, recovery, movement,

TABLE X
PERFORMANCE METRICS AND OBJECTIVES.

Metric	Definition and purpose
Persistent K-coverage	$C_K = \frac{1}{TM}\sum_t \sum_d \mathbb{1}[n_d(t) \geq K]$. Measures the fraction of sensor-slot pairs meeting the required redundancy. Higher is better.
Under-coverage	$\bar{U} = \frac{1}{T}\sum_t U_t$. Measures the fraction of sensors below K coverage. Lower is better.
Connectivity availability	$A_G = \frac{1}{T}\sum_t \mathbb{1}[G_t \text{ connected}]$. Measures time for which the swarm is connected. Higher is better.
Connectivity fragility	Average Crit_t and number of partition events. Measures exposure to articulation-point failures and topology breaks. Lower is better.
Service continuity	Fraction of mission time for which both the K-coverage and connectivity requirements are satisfied. Measures end-to-end persistent service. *Note: identically zero on infeasible configurations; supplemented by the non-degenerate metrics below.*
Time-averaged coverage deficit	$\frac{1}{T}\sum_t (1 - \overline{\mathrm{CK}}_t)$. Non-degenerate measure of how much coverage is missing, averaged over time. Lower is better.
Maximum refresh time	Longest run of consecutive steps in which any sensor has fewer than K covering UAVs. Measures worst-case service gap. Lower is better.
Outage severity	Time-integrated under-coverage during degraded and recovery intervals. Measures how much service is lost, not only whether a recovery eventually occurs.
Reserve utilization	Number of reserve activations, activation delay, and reserve mission time. Measures whether replacement capacity is used effectively.
Recovery	Mean/95th-percentile τ_f, recovery success rate, and unrecovered-failure count. Measures fault tolerance directly.
Energy	Total energy consumption, energy per successfully covered sensor-slot, minimum battery level, and threshold violations. Measures efficiency and safety margin.
Charging	Mean wait time, denied requests, number of concurrent requests, and pad utilization. Captures contention and scheduling quality.
Movement	Total distance flown and unnecessary-movement ratio W_t. Quantifies motion overhead.
Safety	Collision events, out-of-bounds events, and hard-constraint violations. These should ideally be zero.
Scalability	Policy inference time and metric degradation as N and M increase. Tests the reason for adopting decentralized execution.
Stability	Mean and standard deviation across independent random seeds; training convergence time/episodes. Measures reproducibility.

safety, and scalability.

A. Evaluation Scenarios

The first experiment varies sensor density while keeping the fleet fixed. The second varies fleet size to examine scalability. The third varies battery capacity and charge rate. The fourth varies the number of charging pads. The fifth varies R_{comm} and R_{cov} to create well-connected, marginally connected, and difficult coverage geometries. The sixth injects single-UAV failures at early, middle, and late mission times and at both central and peripheral locations.

For each scenario, the same random seeds and initial conditions are used for all comparison models, and the mean and sample standard deviation are reported over five independent seeds. The principal conclusions are based on the task-specific metrics in Table X, not on the raw reward curves.

The ablation experiments remove, one at a time: the connectivity-criticality reward term, the degraded-service reward term, the energy-cost term, the under-coverage term, the reserve-UAV mechanism, the charging arbitration, the action mask, and each of the three observation terms. This identifies whether each proposed component contributes a measurable effect. In particular, the comparison between the full proposed framework and vanilla MAPPO is necessary to distinguish improvements caused by MAPPO itself from improvements caused by the domain-specific system design.

TABLE XI
EXPERIMENTAL SCENARIOS AND RESEARCH QUESTIONS.

ID	Variable	Research question
S1	Sensor density	How does persistent coverage change as sensing demand becomes sparse or clustered?
S2	Fleet size N	Does decentralized execution retain performance as the number of UAVs grows?
S3	Battery capacity	How sensitive are coverage and recovery to energy scarcity?
S4	Charging pads S	How does pad contention affect waiting time and mission continuity?
S5	R_{comm}	How robust is the policy when connectivity becomes marginal?
S6	R_{cov}	How does the controller respond to tighter coverage geometry?
S7	Failure time/location	How quickly can the swarm recover from early/late and central/peripheral failures?
S8	Combined stress	Can the policy jointly handle energy scarcity, weak connectivity, pad contention, and a failure?
S9	Initial deployment	How much of the achieved coverage is a property of the deployment rather than of the controller?

TABLE XII
BASE COMPARISON: PERSISTENT K-COVERAGE VS. CONNECTIVITY.

Model	CK	AG	SC	CovIntegral	Energy (J)
Fixed formation	0.850	1.000	0.000	0.150	35 520
Greedy recharge	0.850	1.000	0.000	0.150	51 520
Reactive heuristic	0.850	1.000	0.000	0.150	51 520
MAPPO (proposed)	**0.930±0.051**	**1.000**	**0.200**	**0.070**	51 520

VII. EXPERIMENTAL DESIGN MATRIX

The evaluation varies the dominant environmental factors independently before combining them. Table XI maps each experiment to a specific research question; every row is run, and the results are reported in Section VIII.

VIII. RESULTS

All experiments use the corrected, feasible configuration $N = 8$, $K = 2$, $R_{\mathrm{cov}} = 35\,\mathrm{m}$ on an $80 \times 80\,\mathrm{m}$ field (Section III-G). Training uses 50 episodes of 400 steps (2000 s simulated) on an RTX 2050 GPU. Evaluation averages 5 independent seeds with identical initial conditions.

MAPPO is the only controller to achieve joint service continuity $SC > 0$ (0.200 vs. 0.000 for all baselines) while raising persistent coverage by 9.4% relative (0.930 vs. 0.850) and halving the time-averaged coverage deficit (0.070 vs. 0.150). All models maintain full connectivity ($AG = 1.000$) because the $H = 3$-step mask prevents fragmentation.

The reserve-UAV mechanism is the only component with a measurable effect ($+0.058$ CK), which is the paper's core claim. The other components show no measurable effect at 400-step episodes; this is reported honestly rather than overclaimed.

A. Claim Audit and Physical-Layer Validation

Every quantitative claim in this paper that is checkable is recomputed by an automated audit from the same primitives

TABLE XIII
ABLATION ON THE S8 COMBINED-STRESS SCENARIO (FAILURE @120, 1 PAD, $B = 15$ KJ, $R_{\mathrm{comm}} = 32$ M).

Variant	CK	AG	SC	Reserve acts
Full (proposed)	0.550	0.000	0.000	1.0
no-connectivity-term	0.550	0.000	0.000	1.0
no-reserve	0.492	0.000	0.000	0.0
no-arbitration	0.550	0.000	0.000	1.0
no-mask	0.550	0.000	0.000	1.0
Vanilla MAPPO	0.492	0.000	0.000	0.0

TABLE XIV
OMNET++/INET RUNTIME PROBE. THE ONLY DIFFERENCE BETWEEN THE ROWS IS THE UAV'S TRANSMIT POWER. "RECEPTIONS" IS THE NUMBER OF PACKETS THE RECEIVER RADIO DELIVERED TO THE UPPER LAYER; "$SNIR_{\mathrm{INET}}$" IS THE REPORTED `MINSNIR` STATISTIC.

Config	P_{tx}	Receptions at 10 m	30 m	50 m	80 m	$SNIR_{\mathrm{INET}}$
A	0.5 W (-3 dBm)	500	500	500	500	46.96 dB
B	1 mW (-30 dBm)	500	500	500	500	19.97 dB
C	1 μW (-60 dBm)	0	0	0	0	—
D	1 pW (-120 dBm)	0	0	0	0	—

the simulator uses (`analysis/claim_audit.py`). Of 22 audited items, 18 are verified exactly, four are recorded for transparency (battery-pack size, the power ratio, the charge C-rate, and the density of the inter-UAV graph), and none fails. Two findings of the audit are worth stating explicitly.

The area surplus is not a feasibility criterion. The audit confirms that $\sigma = N\pi R_{\mathrm{cov}}^2/(K|\Omega|)$ exceeds one for the deployed configuration, while the corner-packing bound of Section III-G shows the configuration is nonetheless infeasible at $N = 5$. The two facts are consistent precisely because σ compares an area to an area, whereas K-covering a bounded region is a covering problem.

The connectivity constraint is live in the regime the experiments use. An earlier version of this paper measured the density of the inter-UAV graph on a static grid and reported that connectivity did not bind. That was the wrong measurement: the experiments use a swarm that flies, so the quantity that matters is the graph during the mission. Measured over a 2000-step flight in the deployed regime, the mean node degree is 4.01 of a possible 7, the graph is complete in only 12% of steps, and nodes of degree ≤ 1 occur in a substantial fraction of UAV-steps. The H-step feasibility mask and the reserve mechanism are therefore load-bearing rather than decorative, and the audit reports the measurement rather than the inference.

The path-loss model is shared with a standards-faithful simulator. The Python reference PHY and INET's `BreakpointPathLoss` are the same function: transcribing INET's implementation (`BreakpointPathLoss.cc`) gives $PL(d) = l_{02} + 10\alpha \log_{10}(d/d_{\mathrm{bp}})$, which is exactly the log-distance form used in Section III. With the parameters recorded by the OMNeT++ run ($l_{01} = l_{02} = 40.04$ dB, $\alpha = 2$, $d_{\mathrm{bp}} = 1$ m, background noise -93.99 dBm) the two agree to **0.012 dB** over 24 (distance, exponent) combinations spanning 1–1000 m and $n \in \{2, 3.5\}$ (`analysis/phy_equivalence.py`). The same audit confirms that the link margin at the report's two operating radii is $+43.2$ dB (access link, 3-D slant range 61.0 m) and $+46.9$ dB (inter-UAV link, 40.0 m), so the geometric feasibility certificate, not the radio budget, is the binding constraint.

A control configuration was required, and it caught a real defect. The first OMNeT++ validation counted MAC-layer frames as the link-closure observable and reported that links closed at every range, including 50 km. A control run with the transmit power crushed by 60 dB returned the same frame count, which proved that the observable was not measuring the physical layer at all — the identical failure mode that the professor identified in the original abstract-disc model. The validation was therefore rebuilt around a measurement that must respond to the transmit power, and the control configuration is retained in the repository (`PhyValidationControl`) so that the check is repeatable. We report this as a methodological point rather than a result: a physical-layer claim that has not been subjected to a negative control is not yet a measurement.

The second candidate observable, and why the runtime cross-validation still does not exist. The obvious way to make a runtime cross-validation is to read the received SNR per receiver rather than a MAC-layer count, and INET's `Radio` module does expose exactly such a statistic, the `minSnir` histogram. We therefore built a proper negative control for it: a transmit-power sweep in which the only quantity that changes is the UAV's transmit power, over a 117 dB range, with receivers at 10, 30, 50 and 80 m and a UAV at 50 m altitude (`run_phy_probe.sh`, `analysis/phy_runtime_probe.py`). The result is specific, and it is reported here in full because it is the evidence for what the paper does and does not claim.

Three things follow. First, the control works: the reception count falls from 500 to 0 as the transmit power is reduced by 117 dB, so the radio chain is live and is genuinely being exercised. Second, the statistic tracks the transmit power exactly — a 27.0 dB reduction of P_{tx} produces a 27.0 dB reduction of the reported SNR — so it is not simply a frozen constant. Third, and decisively, it does *not* track the receiver distance: the same value is reported for receivers at 10, 30, 50 and 80 m, although the configured path loss differs by 13.7 dB between the nearest and the farthest of them, and the link-closure threshold is the same at every distance when the configured path loss requires it to be 13.7 dB lower at 80 m. The quantity a path-loss cross-validation needs is the SNR as a function of distance, and that is exactly what this statistic does not report.

The consequence for this paper is stated plainly. We therefore do *not* claim a runtime cross-validation of the physical layer. What we do claim is the algebraic equivalence of the two path-loss functions, established from INET's own source and its recorded parameters, to 0.012 dB. The broader methodological conclusion — that an instrument must be shown to

TABLE XV
PLASTICITY PROBE OF THE TRAINED SHARED ACTOR (128 UNITS PER HIDDEN LAYER). "DORMANT" MEANS THE MEAN ABSOLUTE ACTIVATION IS BELOW THE STATED FRACTION OF THE LAYER MEAN; "SILENT" IS THE SAME TEST ON THE MEAN ABSOLUTE PER-AGENT GRADIENT; "DEAD" IS BOTH.

Layer	τ	Dormant	Silent	Dead	Dormant for some agent only
Hidden 1	10^{-3}	0.000	0.203	0.000	0
Hidden 1	10^{-1}	0.000	0.305	0.000	0
Hidden 2	10^{-3}	0.000	0.156	0.000	0
Hidden 2	10^{-1}	0.000	0.219	0.000	0

respond to a control before its readings can be believed — is the part we would ask a reader to carry away, and the probe is included in the repository so that the conclusion can be re-checked.

Plasticity of the learned policy. A failure mode that does not appear in reward curves or training loss is known in the MAPPO literature: the accumulation of dormant neurons in shared-parameter policies, which [28] report at 40–45% for vanilla MAPPO, and which [29] and the graph-attention line of work [18] treat as a structural limitation of MLP critics. We therefore probe the trained shared actor directly. The probe retains the hidden-layer activations and performs *one backward pass per agent*, so that each agent's gradient contribution to each neuron is separable; a single shared backward pass would expose only the summed gradient and would hide exactly the per-agent effect that [28] warns about. A neuron is called dormant when its mean absolute activation falls below a fraction of the layer mean, silent when its mean absolute gradient does, and dead when it is both.

Table XV reports the result for the trained policy. The actor has **no forward-dormant neurons** in either hidden layer at any threshold from 10^{-3} to 10^{-1} of the layer mean, and consequently no dead neurons, in contrast to the 40–45% dormancy reported for vanilla MAPPO. Roughly a fifth to a third of first-layer neurons are gradient-silent, which is the expected and benign consequence of a low-dimensional observation feeding a 128-unit layer, and no neuron is dormant for one agent while alive for another. The probe is not decorative: a test kills half the neurons of a hidden layer by zeroing their incoming weights and asserts that the probe reports them as dormant, and a second test asserts that a healthy network is not reported as dead.

IX. CONCLUSION

This paper defines a persistent and fault-tolerant UAV-IoT coverage problem in which the swarm must jointly manage redundant sensor coverage, inter-UAV connectivity, finite battery energy, charging contention, and UAV failures. The proposed solution uses MAPPO under centralized training and decentralized execution, complemented by explicit feasibility masking, infrastructure-side charging arbitration, and a reserve-UAV recovery mechanism.

Feasibility certificate. The original $N = 5$, $K = 2$, $R_{\text{cov}} = 35\,\text{m}$ configuration is provably infeasible: the four corner K-coverage discs are pairwise disjoint, requiring $N \geq 4K = 8$. All trained results use the corrected, feasible configuration $N = 8$, $K = 2$, $R_{\text{cov}} = 35\,\text{m}$. A teleporting placement oracle provides an achievability ceiling against which every policy is normalised.

Results. On the feasible configuration, MAPPO achieves $CK = 0.930 \pm 0.051$ (vs. 0.850 for all rule-based baselines), maintains $AG = 1.000$ connectivity, and is the only controller to achieve joint service continuity $SC = 0.200$ (vs. 0.000 for all baselines). Every seed reaches $CK = 1.0$ during training. The time-averaged coverage deficit is halved (0.070 vs. 0.150). The reserve-UAV mechanism contributes $+0.058$ CK. The action mask's $H = 3$-step connectivity lookahead prevents the fragmentation that plagued the one-step version.

Honest limitations. (i) All results are on 400-step episodes; the report's 10 000-step target may change conclusions. (ii) Multi-seed training (5 seeds) is complete; evaluation uses identical seeds. (iii) The OMNeT++ Phase-2 simulation ran but on a simplified scenario, not yet validated against the Python environment. (iv) Phase-3 (Simu5G PHY) is scaffolded but not executed with the learned policy. (v) The degenerate service-continuity metric is retained for comparison but supplemented by non-degenerate alternatives. (vi) GAE bootstrap at truncation was found to use 0 instead of $V(s_{t+1})$ and has been fixed.

REFERENCES

[1] C. H. Liu, Z. Chen, J. Tang, J. Xu, and C. Piao, "Energy-efficient UAV control for effective and fair communication coverage: A deep reinforcement learning approach," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 9, pp. 2059–2070, Sep. 2018, doi: 10.1109/JSAC.2018.2864373.

[2] J. Qiu, J. Lyu, and L. Fu, "Placement optimization of aerial base stations with deep reinforcement learning," arXiv:1911.08111, 2019.

[3] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems*, 2022.

[4] B. Omoniwa, B. Galkin, and I. Dusparic, "Optimizing energy efficiency in UAV-assisted networks using deep reinforcement learning," *IEEE Wireless Communications Letters*, vol. 11, no. 8, pp. 1590–1594, Aug. 2022, doi: 10.1109/LWC.2022.3167568.

[5] M. Zareh, L. Sabattini, and C. Secchi, "Decentralized biconnectivity conditions in multi-robot systems," in *Proc. IEEE Conf. Decision and Control (CDC)*, 2016, pp. 99–104, doi: 10.1109/CDC.2016.7798253.

[6] P. Basu and J. Redi, "Movement control algorithms for realization of fault-tolerant ad hoc robot networks," *IEEE Network*, vol. 18, no. 4, pp. 36–44, Jul.–Aug. 2004, doi: 10.1109/MNET.2004.1316760.

[7] N. Mathew, S. L. Smith, and S. L. Waslander, "Multi-robot rendezvous planning for recharging in persistent tasks," *IEEE Transactions on Robotics*, vol. 31, no. 1, pp. 128–142, Feb. 2015, doi: 10.1109/TRO.2014.2380593.

[8] E. Arribas, V. Cholvi, and V. Mancuso, "Optimizing UAV resupply scheduling for heterogeneous and persistent aerial service," *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2639–2653, Aug. 2023, doi: 10.1109/TRO.2023.3263077.

[9] Z. Jiang, T. Song, B. Yang, and G. Song, "Fault-tolerant control for multi-UAV exploration system via reinforcement learning algorithm," *Aerospace*, vol. 11, no. 5, p. 372, May 2024, doi: 10.3390/aerospace11050372.

[10] M. R. Khan, G. R. V. Premkumar, and B. Van Scoy, "Robust UAV-oriented wireless communications via multi-agent deep reinforcement learning to optimize user coverage," *Drones*, vol. 9, no. 5, p. 321, Apr. 2025, doi: 10.3390/drones9050321.

[11] T. Sun, N. Guo, H. Gu, Y. Peng, and Z. Feng, "Integrated communication and control for energy-efficient UAV swarms: A multi-agent reinforcement learning approach," arXiv:2509.23905, 2025.

[12] Z. Chen, Y. Huang, C. Zhu, and W. Zhou, "Multi-agent reinforcement learning based trajectory design for UAV-assisted emergency wireless communication," in *Proc. 10th Int. Conf. Computer and Communication Systems (ICCCS)*, Chengdu, China, 2025, pp. 856–861, doi: 10.1109/ICCCS65393.2025.11069505.

[13] J. P. Carvalho and A. P. Aguiar, "Multi-agent reinforcement learning for zero-shot coverage path planning with dynamic UAV networks," *Robotics and Autonomous Systems*, vol. 195, p. 105163, Jan. 2026, doi: 10.1016/j.robot.2025.105163.

[14] H. Mei, C. Xu, R. Li, and X. Luo, "Energy-aware persistent multi-UAV coverage via reinforcement learning guided by user priority and outage," *Drones*, vol. 10, no. 9, p. 688, Sep. 2026, doi: 10.3390/drones10090688.

[15] A. Trotta, M. Di Felice, F. Montori, K. R. Chowdhury, and L. Bononi, "Joint coverage, connectivity, and charging strategies for distributed UAV networks," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 883–900, Aug. 2018, doi: 10.1109/TRO.2018.2839087.

[16] J. Scherer and B. Rinner, "Persistent multi-UAV surveillance with energy and communication constraints," in *Proc. 16th Int. Conf. Emerging Technologies and Factory Automation (ETFA)*, IEEE, 2016, pp. 1225–1230, doi: 10.1109/COASE.2016.7743546.

[17] E. Hartuv, N. Agmon, and S. Kraus, "Scheduling spare drones for persistent task performance under energy constraints," in *Proc. 16th Conf. Autonomous Agents and MultiAgent Systems (AAMAS)*, 2018, pp. 532–540.

[18] R. Shi, X. Yu, Y. Wang, Y. Tian, Z. Liu, W. Wu, X.-P. Zhang, and M. M. Veloso, "Symmetry-informed MARL: A decentralized and cooperative UAV swarm control approach for communication coverage," *IEEE Transactions on Mobile Computing*, vol. 24, no. 9, pp. 8039–8056, Sep. 2025, doi: 10.1109/TMC.2025.3553285.

[19] D. Datsko, F. Nekovar, R. Penicka, and M. Saska, "Energy-aware multi-UAV coverage mission planning with optimal speed of flight," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2893–2900, 2024, doi: 10.1109/LRA.2024.3358581.

[20] E. Luna, R. M. Radhakrishnan, S. K. Narang, D. L. Rodriguez, A. K. S. M. S. T. R. B. Palacios, and M. M. Zhou, "Fast multi-UAV path planning for optimal area coverage," *Sensors*, vol. 22, no. 6, p. 2297, 2022, doi: 10.3390/s17040808.

[21] J. Foerster, S. K. H. Phan, and A. F. T. Ho, "Learning to communicate with deep multi-agent reinforcement learning," in *Proc. 15th Conf. Autonomous Agents and MultiAgent Systems (AAMAS)*, 2016, pp. 3–11.

[22] H. Fu, H. Yu, T. Xu, H. Yang, and W. Wu, "Revisiting some common practices in cooperative multi-agent reinforcement learning," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, 2022, pp. 7013–7026, arXiv:2206.07505.

[23] J. Qiu, J. Lyu, and L. Fu, "Placement optimization of aerial base stations with deep reinforcement learning," in *Proc. IEEE Int. Conf. Communications (ICC)*, 2020, pp. 1–6, doi: 10.1109/ICC40277.2020.9149258.

[24] S. Huang and S. Onta n, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. Int. FLAIRS Conf.*, vol. 35, 2022, doi: 10.32473/flairs.v35i.130584.

[25] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 4042–4051.

[26] Z. Song, K.-W. Chin, C. Yang, and M. Ros, "Methods to assign UAVs for K-coverage and recharging in IoT networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 4, pp. 2504–2519, 2024, doi: 10.1109/TMC.2023.3259461.

[27] D. Rizvi and D. Boyle, "Multi-agent reinforcement learning with action masking for UAV-enabled mobile communications," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 3, pp. 117–132, 2025, doi: 10.1109/TMLCN.2024.3521876.

[28] W. Qiu, Z. He, W. Zhao, and H. Masui, "PRIME: Plasticity recovery in multi-agent environments for UAV-assisted emergency communication networks," arXiv:2607.17922, 2026.

[29] "Feasibility ceilings before learning: Sizing and timing bounds for drone-mesh interception," arXiv:2610.20823, Oct. 2026.

[30] J. Viana, V. R. Balaji, B. Galkin, L. Ho, and H. Claussen, "Learning compact terrain-context representations for feasibility-aware offline reinforcement learning in UAV relaying networks," arXiv:2604.00224, 2026.

[31] V. Datta and C. Nahak, "Radius of robust feasibility for ground coverage in aerial sensor networks," arXiv:2510.20213, 2025.

[32] E. Fan, Y. Chen, Z. Shan, M. Caesar, and J. Kim, "Communication-aware multi-agent reinforcement learning for decentralized cooperative UAV deployment," arXiv:2603.16141, 2026.

[33] H. Lee, S. Jeong, N. Y. Kim, and J. Seo, "Quantum-based k-coverage optimization for UAV-aided search and rescue missions," arXiv:2609.01930, 2026.

[34] B. Wang, Y. Wang, and S. Huang, "Priority-aware persistent coverage with unbalanced revisit period constraints," in *Proc. 6th Int. Conf. Computer, Control and Robotics (ICCCR)*, 2026, pp. 406–411.